Knowledge Distillation and Deployment of a Command-Parsing Model on a Local Cloud

S Naveen — 230101100 — CS3081

Repository: https://github.com/Sukirti08/kd-command-parser · **Tag:** `final` · **Machine:** Intel i5-1155G7 (4 cores/8 threads), 8 GB RAM, Windows 11 + WSL2 Ubuntu 24.04, Docker Desktop 4.98, k3d 5.9 (all measurements on this machine)

Architecture

Laptop (Ollama + phi3.5) → k3d cluster `kd` (1 server node). Namespaces: `data` (SeaweedFS, S3 API on 8333, buckets `datasets`, `models`), `pipeline` (sharded teacher Jobs), `app` (parser Deployment, Service, Traefik Ingress `/parse`, HPA). The model is never in the image; Pods fetch `MODEL_URI` from S3 at start-up using credentials from a Secret.

`Ollama(phi3.5, laptop)` ← `pipeline: teacher-worker-0/1` → `data: SeaweedFS (datasets/teacher/shard-*.jsonl)` → `clean.py → datasets/v1/train.jsonl` → `training (CPU)` → `models/v1, v2` + GitHub Release → `app: parser Pods (download at start-up)` ← `Service` ← `Traefik Ingress /parse` ← clients / Locust

Task 1 — Object storage and parallel teacher labelling

Setup. Cluster from `cluster/k3d.yaml`; SeaweedFS in `data`; `commands_pool.jsonl` in `datasets`. Teacher: `phi3.5` on the laptop via Ollama, `temperature=0, seed=42`, `num_predict=64` and stop sequences `"\nCommand"`, `"\n\n"`, prompt `prompts/teacher_prompt.txt` (rules + one example per intent, none from validation).

Sharding. Each worker reads `SHARD_INDEX` (from 0) and `SHARD_COUNT` and keeps line n of the pool iff n mod `SHARD_COUNT` = `SHARD_INDEX`. The residues 0…k−1 partition the integers, so every line belongs to exactly one shard: no gaps and no overlaps by construction (verified by `worker/check_shards.py`, `results/shard_check.txt`). Modulo (interleaved) assignment also balances shards to within one record and mixes command types evenly, unlike contiguous ranges. Each worker writes only its own object `datasets/teacher/shard-<i>.jsonl`, so workers never contend for a write. The object is re-uploaded every 10 records and a restarted worker skips ids already present, so a Job retry resumes rather than restarts. Limitation: both workers share one Ollama process on one CPU, so throughput barely improves with more shards; sharding pays off only with several teacher replicas on separate cores or machines. MEASURED throughput: about 0.04–0.05 commands/s per worker (≈11–14 s per command in total while the laptop was active); total wall-clock time ≈ 25 h, including the overnight throttling and the restart described below.

What happened in practice (MEASURED). On this 8 GB CPU-only laptop phi3.5 first took about 35 s per command because it kept generating extra few-shot examples after the JSON until `num_predict`; adding stop sequences cut this to about 11 s per command without changing the JSON. While the laptop sat idle overnight Windows throttled the CPU (about 5 commands per hour against about 250 per hour when active), which was fixed by disabling power throttling for Ollama and keeping the machine awake. A Windows restart stopped both worker Jobs (they exhausted their retries); after recreating them with `backoffLimit: 100` they resumed from the 730 records already saved in object storage, demonstrating the resumable, per-shard design.

Cleaning counts (MEASURED, `results/clean_counts.json`; checks applied in order a→d, each record counted once, de-duplication by normalised text after filtering):

Input	(a) invalid JSON	(b) wrong keys	(c) bad intent	(d) slot not in command	Duplicates	Kept
3000	3	4	2	228	43	2720

Check: input = a + b + c + d + duplicates + kept. SHA-256 of `train.jsonl`: `aa00c121920513f17f04665d98fba7a6156a7c46ace6603f64d885ad64fcbc89` (`datasets/v1/train.sha256`). Decisions: slot match is case-insensitive with whitespace collapsed; empty strings fail (d); `teacher_raw` is stored byte-for-byte, and `target` is the same JSON re-serialised compactly in key order (a deterministic transformation, not a manual edit).

Task 2 — Training the student

`training/config.yaml`: seed 42, lr 3e-4, batch 16, 8 epochs, max length 96, versions pinned. Example format `Command: <text>\nJSON: <target><eos>`; loss on the JSON tokens only. v2 = 10 epochs. Loss per epoch (`training/loss.csv`, `training/loss_v2.csv`): v1 0.601842 → 0.01383 over 8 epochs; v2 0.624754 → 0.007555 over 10 epochs. Training time MEASURED: v1 57 min on the CPU; v2 took 11.8 h because it ran overnight while Windows throttled the idle laptop and, for several hours, alongside an accidental second copy of the same training job (found with `ps` and stopped). **Suitability of Pythia-70M.** A 70M-parameter base model (2,048-token context, trained on the Pile, not instruction-tuned) is far too small for open-ended language tasks, and untrained it cannot produce the format at all (0% JSON validity below). For this task, however, the output is a short fixed template and the slots are copied from the input, which suits a small causal LM: fine-tuning on 1,500 examples was enough to reach 100% validity. Its weak point is semantics: confusing similar intents and choosing slot boundaries. It is also small enough to train on a laptop CPU and to serve within a 1 CPU / 1 GiB container.

Model (validation, 100)	json_validity	intent	date	time	place	full_frame
Teacher phi3.5	100.0	84.0	74.0	83.0	86.0	42.0
Pythia-70M untrained	0.0	0.0	0.0	0.0	0.0	0.0
Student v1	100.0	83.0	80.0	86.0	86.0	48.0

Fine-tuning turned a model with 0% validity into one with 100%: the student learned the output template completely. Its full-frame accuracy (48%) is close to, and here slightly above, the teacher's (42%). Two likely reasons: cleaning removed 228 teacher replies with slot values not present in the command, so the student learned only from the teacher's cleaner outputs; and with 100 validation examples each example is worth 1 point, so a difference of a few points is within noise. MEASURED student errors (`results/student_v1_predictions.jsonl`): "remove the upcoming calendar event": date gold `None` vs predicted `upcoming`; "need an alarm set for sunday at five in the morning": time gold `five` vs predicted `five in the morning`; "give me the time at washington right now": time gold `None` vs predicted `right now`.

Sequence-level vs logit-based distillation. (a) Sequence-level KD (Kim & Rush, 2016) trains the student with ordinary cross-entropy on the teacher's *generated output sequence*: the teacher's decoded reply is treated as the label for each input, so the student imitates the teacher's mode. (b) Logit-based (token-level) KD trains the student to match the teacher's full *probability distribution over the vocabulary at every position*, typically minimising KL divergence between temperature-softened teacher and student softmaxes, which passes on "dark knowledge" about plausible alternatives. (c) That is not possible here: Phi-3.5 and Pythia-70M use different tokenizers and vocabularies (≈32k vs ≈50k tokens),

so their distributions are over different label sets and cannot be compared position by position; in addition Ollama returns only text, not per-token logits. (d) Limitation: the student only ever sees the teacher's single greedy answer, so it inherits the teacher's systematic errors (e.g. "remove the upcoming calendar event": date gold `None` vs predicted `upcoming`) with no signal about the teacher's uncertainty, and the cleaning step removes only *invalid* replies, not *wrong but valid* ones.

Task 3 — Inference service in a container

FastAPI on port 8000: `POST /parse`, `GET /healthz`, `GET /readyz` (503 until the model is loaded). Config only from `MODEL_URI`, `MODEL_VERSION`, `S3_ENDPOINT`, `MAX_NEW_TOKENS`, `FAULT_INJECT`; credentials from a Secret. Every output is validated against `schema.json`; invalid → HTTP 500 `{"error":"invalid_model_output"}`. Greedy decoding stops at the end of the first JSON object. One generation at a time per Pod (lock) and `torch` set to 1 thread to match the 1-CPU limit. Image: `python:3.11-slim`, CPU-only torch, all packages pinned, size MEASURED 1.27 GB on disk (273 MB compressed). 50 sequential requests at `--cpus 1 --memory 1g`: 50 sequential requests in 39.6s, status codes: {200: 50}; peak memory MEASURED 596.2 MiB of process memory (`docker stats`). The cgroup `memory.peak` reached the 1 GiB limit because it also counts reclaimable page cache from the downloaded model files; no process was killed: OOMKilled = false, RestartCount = 0 (`results/task3_memtest/`).

Task 4 — Kubernetes deployment, scaling and reliability

Manifests in `k8s/app/`. Requests 500m CPU / 512Mi, limits 1 CPU / 1Gi. Readiness `/readyz` (2 s), liveness `/healthz`. RollingUpdate `maxSurge 1, maxUnavailable 0` + 8 s `preStop` so endpoints are removed before shutdown. Endpoints empty while loading: `/readyz` returns 503 until the model is loaded, so a loading Pod is not added to the Service. MEASURED evidence: in a first recovery attempt with only one replica, Traefik answered 404 to every request until the replacement Pod became Ready (no ready endpoint behind the route); with two replicas the same test had 0 failures.

Test	MEASURED result	Evidence
HPA 1→2→1	scaled 1 → 2 under an 8-user load (CPU above 70% of request) and 2 → 1 after the load stopped (SuccessfulRescale events)	`results/task4_hpa/`
Rolling update v1→v2 + undo	1/297 = 0.34% failed (one HTTP 502 while a Pod terminated; an earlier attempt with 1 replica had 4/331 = 1.2%); header after undo v1. The script hung on a header check without a timeout, so `kubectl rollout undo` was run by hand after the load test	`results/task4_rolling/`
Recovery (delete 1 Pod)	replacement Pod created and Ready, Deployment back to 2/2; failures 0/286 = 0.0%	`results/task4_recovery/`
Cold start ×3	49 s, 54 s, 24 s; median 49 s (model download 6–37 s per run)	`results/task4_coldstart/`
Load balancing (2 replicas)	7hp49 91 (50.3%), d8lrb 90 (49.7%)	`results/task4_loadbalance/`

Limitations. Single node: no fault tolerance across machines; a laptop failure takes down the whole cluster, storage included (SeaweedFS has one replica on a local-path volume). HPA uses metrics-server samples (~15 s) and CPU % of *request*, and max 2 replicas on shared laptop cores means a second replica adds little real capacity. Host port 8080 was occupied by a Windows service (AgentService.exe), so on

this laptop the Ingress was published on 8081 through a git-ignored `local.mk`; the repository default remains 8080.

Task 5 — Subnetting and network ACL

Table and working: `network/subnets.md`. app 10.20.0.0/22, public 10.20.4.0/24, data 10.20.5.0/25, management 10.20.5.128/27. NACL rules: `network/make_nacl.py` → `nacl.json`; `nacl_sim.py --check`: 6/6 (100%). **Why:** a NACL is stateless and does not remember connections, so every reply is judged as a new packet and needs its own rule (clients' ephemeral ports 1024–65535 outbound; app replies to the LB's ephemeral ports inbound). Rules are evaluated lowest number first and the first match wins, so the deny for 198.51.100.0/24 (rule 90) must be numbered below the allows for 80/443 (100/110); otherwise the allow would match first and admit the abusive range.

Task 6 — Reproducibility, testing and CI/CD

`make up` (3 commands from a fresh clone), `make down`. Workflow `.github/workflows/ci.yml`: lint (ruff) → unit tests (13) → evaluation (Release asset, checksum verified, `score.py`) → quality gate (`--min-validity 95`); gitleaks in parallel. Passing run: https://github.com/Sukirti08/kd-command-parser/actions/runs/42474155821. **Failing gate run (untrained Pythia-70M):** https://github.com/Sukirti08/kd-command-parser/actions/runs/42466398400, validity 0% < 95%. Gitleaks: no leaks.

Task 7 — Performance, cost and review

MEASURED (standard Locust command, 2 replicas, `results/task7_perf/`): p50 2700 ms, p95 8200 ms, throughput 1.27 req/s, error rate 0.0 % (0/151), per Pod (d8lrb: CPU 671m avg / 930m max, memory 414 MiB avg / 424 MiB max; sgkw8: CPU 731m avg / 898m max, memory 349 MiB avg / 372 MiB max), cold start 49 s.

Cost (rates ASSUMPTION from `rate_card.json`): per replica-hour = 1 vCPU × ₹3.5 + 1 GiB × ₹0.45 = ₹3.95; 2 replicas = ₹7.90/h; requests/h = 1.270 × 3600 = 4571.7; cost per 1,000 = 7.90 / 4571.7 × 1000 = **₹1.73**.

SLO: 99% of `/parse` requests succeed (HTTP 200 within 30 s), window = one standard load-test run. Requests 151 → error budget = 1% × 151 = 1.51 requests; failures 0 → 0% of the budget used.

Concern	Evidence (measured)	Limitation found	Improvement
Data and storage	2720/3000 kept (9.3% removed; 228 invented slots)	One SeaweedFS replica on a local volume; a laptop restart stopped the labelling Jobs	Replicated object storage; a higher Job backoffLimit from the start (resumable workers already avoided data loss)
Model quality	full frame student 48% vs teacher 42%; validity 100%	Student copies teacher mistakes; only 100 validation examples, so differences of a few points are noise	Larger validation set; filter wrong-but-valid replies (e.g. teacher self-consistency)
Latency and throughput	p95 8200 ms, 1.27 req/s	One generation at a time per Pod on 1 CPU; both Pods share 4 laptop cores	Batch requests, quantise the model (int8), shorter MAX_NEW_TOKENS
Resource utilisation	CPU 671–731m avg, 350–415 MiB per Pod	CPU-bound at the 1-core limit while memory used	Lower the memory limit, raise the CPU limit or the

Concern	Evidence (measured)	Limitation found	Improvement
		under half of 1 GiB	replica count
Reliability and recovery	rolling 0.34% failed, recovery 0.0% failed, cold start median 49 s	Single node: no fault tolerance across machines; 1 replica gives downtime during updates	Multi-node cluster, minReplicas ≥ 2, a PodDisruptionBudget
Security and cost	₹1.73 per 1,000 requests; gitleaks: no leaks	Credentials are a static S3 key in a Secret; cost dominated by idle CPU at low traffic	Short-lived credentials (IRSA-style), scale to zero when idle

References

Kim & Rush, EMNLP 2016; Chen et al., ACL 2020; Bastianelli et al., SLURP, EMNLP 2020; Biderman et al., Pythia, ICML 2023; documentation of k3d, SeaweedFS, Kubernetes, Hugging Face Transformers.